Final Review - ISTQB CTFL v4.0 Cheat Sheet

Exam: 40 questions · 60 minutes (+15 if English is not your first language) · pass 65% = **26/40** · no negative marking · all 6 chapters examinable at K1 (any keyword can be tested) · distribution: Ch1=8, Ch2=7, Ch3=4, Ch4=13, Ch5=6, Ch6=2

1. The Seven Testing Principles

#	Principle	Exam cue
1	Testing shows the presence, not the absence of defects	Never proves correctness
2	Exhaustive testing is impossible	Use techniques + prioritization + risk-based testing
3	Early testing saves time and money	Static + dynamic start ASAP; early defects cascade if missed
4	Defects cluster together	Pareto; clusters feed risk-based testing
5	Tests wear out	Add/modify tests; automated regression repetition stays useful
6	Testing is context dependent	No universal approach
7	Absence-of-defects fallacy	Verified + fixed can still fail users: add validation

2. Core Vocabulary Chains

Concept	What to remember
Error → defect → failure	Human error makes defect (bug); executing it MAY cause failure. Root cause = fundamental reason; fixing it prevents similar failures
Failure w/o human error	Environmental causes: radiation, electromagnetic fields
Verification vs validation	Verification: meets specified requirements. Validation: meets users' needs in operation
Testing vs debugging	Testing triggers failures / finds defects. Debugging: reproduce → diagnose root cause → fix → confirmation test
QC vs QA	QC product-oriented corrective (testing = major form of QC). QA process-oriented preventive (improves processes)
Static finds directly	No failure needed: no reproduction/diagnosis. Finds unreachable code, non-executable defects
7 test activities	Planning → monitoring & control → analysis (what to test) → design (how to test) → implementation (procedures, suites, env) → execution → completion (consolidate)
Independence	Low→High: author → peer/same team → independent test team → external org. Benefit: outside view. Drawback: isolation, slow feedback

3. Test Levels vs Test Types

Level (5)	Focus / owner	Type (4) - applies at every level
Component (unit)	Component isolated; developer; needs harness	**Functional**: what it does (completeness, correctness, appropriateness)
Component integration	Interfaces between components; bottom-up / top-down / big-bang	**Non-functional**: how well - ISO 25010: performance, compatibility, usability, reliability, security, maintainability, portability
System	Whole system end-to-end; independent team; system specs	
System integration	Interfaces to external systems; production-like environment	**Black-box**: specification-based, behavior vs docs
Acceptance	Validation + deployment readiness; UAT / operational / contractual & regulatory / alpha / beta	**White-box**: structure-based; covers internal structure

Confirmation testing *= original defect fixed (re-run failed tests / add new).* ***Regression testing*** *= change broke nothing else (impact analysis first; automate early; suite grows each iteration). Both needed at all levels where changes happen.*

4. Static Testing and Reviews

Topic	Memory table
Review types ladder	Informal (no process/output) → Walkthrough (AUTHOR leads; educate, consensus, confidence) → Technical review (MODERATOR + qualified reviewers; technical consensus) → Inspection (most formal; full generic process; MAX anomalies; metrics; author ≠ leader/scribe)
Review process (5)	Planning → Initiation (everyone prepared) → Individual review (log anomalies) → Communication & analysis (anomaly ≠ necessarily defect; decide status/owner) → Fixing & reporting
Roles (6)	Manager: decides what + resources · Author: writes/fixes · Moderator: runs meeting, safe environment · Scribe: records · Reviewer: reviews · Review leader: overall, who/when/where
Static finds cheaply	Requirement ambiguities/inconsistencies; design flaws; undefined/undeclared variables; unreachable/duplicate code; standards deviations; interface spec errors; buffer overflows; coverage gaps
Success factors	Clear objectives + measurable exit criteria (NEVER evaluate people); right type; small chunks; feedback given; prep time; management support; culture; training; facilitation

5. Techniques and Coverage - the K3 chapter (13 Qs)

Technique	Coverage items	100% rule / trap
Equivalence Partitioning	The partitions (valid + invalid)	One test per partition; non-overlapping, non-empty; Each Choice = each partition of each set once (no combos)
BVA 2-value	Boundary + 1 closest neighbor each side	Range 1-500 → 0, 1, 500, 501
BVA 3-value	Boundary + both neighbors	Adds 2, 499; catches x<=10 coded as x==10 via 9
Decision table	Feasible columns (rules)	Full table = 2^n rules; T/F; '-' irrelevant; N/A infeasible; minimize by merging
State transition	States / valid transitions (0-switch) / all transitions	all-transitions ⊇ valid-transitions ⊇ all-states; ONE invalid transition per case (fault masking); safety-critical minimum: all transitions
Statement testing	Executable statements	100% does NOT guarantee branches tested
Branch testing	Branches (control-flow transfers)	Subsumes statement coverage (100% branch ⇒ 100% statement, not vice versa)

White-box: *strength = whole implementation considered; weakness = defects of omission.* ***Error guessing****: fault attack lists (input, output, logic, computation, interfaces, data).* ***Exploratory****: design+execute+evaluate simultaneously; session-based = time-box + charter + debrief; best with poor specs / time pressure.* ***Checklist-based****: question items, updated regularly, kept short.*

6. User Stories, Acceptance Criteria, ATDD

Item	Content
3 Cs	Card (medium) · Conversation (how used) · Confirmation (acceptance criteria)
Format	As a [role], I want [goal], so that [business value]
INVEST	Independent · Negotiable · Valuable · Estimable · Small · Testable (can't test it? unclear or not valuable)
Acceptance criteria uses	Define scope · consensus · positive + negative scenarios · basis for acceptance tests · planning/estimation. Formats: scenario (Given/When/Then) or rule (bullet list, I/O table)
ATDD order	Specification workshop → tests before code → POSITIVE first, then negative, then non-functional; tests = executable requirements

7. Managing Test Activities (Ch. 5)

Topic	Content
Entry criteria	Resources (people, tools, env, data, budget); testware (basis, requirements); initial quality (smoke pass). Agile: Definition of READY
Exit criteria	Thoroughness (coverage, unresolved defects, density) + completion (planned tests run, defects reported). Agile: Definition of DONE. Time/budget out = valid exit w/ stakeholder risk acceptance
Estimation (4)	Ratios (historic 3:2; 600 dev → 400 test) · Extrapolation (current-project data) · Wideband Delphi (isolated experts, iterate to consensus; Planning Poker = Agile cards) · Three-point: **E=(a+4m+b)/6, SD=(b-a)/6** (a=6,m=9,b=18 → 10±2)
Prioritization	Risk-based · Coverage-based (+ additional coverage variant) · Requirements-based. BUT dependencies and resource windows override priority
Test pyramid	Bottom: many, small, isolated, fast. Top: few, complex, slow, end-to-end. Higher layer = lower granularity/isolation, longer runtime
Quadrants	Q1 tech/support: component + integration, automate in CI; Q2 business/support: functional, examples, API, simulations; Q3 business/critique: exploratory, usability, UAT (manual); Q4 tech/critique: smoke + non-functional except usability

8. Risk, Reports, Defects, Tools

Topic	Content
Risk level	Likelihood (between 0 and 1, exclusive) × Impact (quantitative) or risk matrix (qualitative)
Project vs product risks	Project: organizational, people, technical, supplier → schedule/budget/scope. Product: quality characteristics (wrong calculations, poor UX, security) → dissatisfaction, revenue loss, penalties, injury
Risk mgmt cycle	Analysis = identification + assessment (categorize, rate, prioritize). Control = mitigation (right testers, independence, reviews, techniques, types, regression; or accept/transfer/contingency) + monitoring
Reports	Progress (periodic): period, deviations, impediments, metrics, new risks, next period. Completion (once): summary, evaluation vs plan, deviations, unmitigated risks, unfixed defects, lessons learned
CM + defect mgmt	CM: items version-controlled + traceable; approved = baseline; revert to reproduce results. Defect report: ID · title · date/author · object + environment · context · repro steps + evidence · expected vs actual · severity · priority · status · references
Tools + automation	Tool types: management, static, design & implementation, execution & coverage, non-functional, DevOps, collaboration, scalability (VMs/containers), even a spreadsheet. Automation BENEFITS: less repetitive work, consistency, objective measures, faster feedback, deeper tests. RISKS: unrealistic expectations, underestimated cost, tool where manual fits, over-reliance, vendor lock, OSS abandonment, platform/compliance mismatch

Last-minute drill: *recite 7 principles · 5 levels · 4 types · 4 review types + leaders · 3 state-transition criteria · both estimation formulas · 2 quadrant axes. Watch stems for NOT / EXCEPT / BEST. ~90 s/question, flag and return, answer everything.*